NOTEBOOK 02B

Python for Actuaries Part 2

Agenda

In this notebook, we will cover:

- Strings
- Lists
- Sets
- Dictionaries

Built-in Data Types in Python

In addition to user-defined classes, Python offers a variety of **built-in data types** that are extremely useful in everyday programming. These data types are also based on classes, which means they have specific methods and properties that we can work with directly.

The most important built-in data types include:

- **Strings**: A sequence of characters that stores text.
- **Lists**: An ordered collection of elements.
- **Sets**: An unordered collection of unique elements.
- **Dictionaries**: A collection of key-value pairs.

These data types are predefined in Python and can be used directly without needing to create them ourselves. They are very flexible and allow us to efficiently solve many tasks.

In the following sections, we will look at how these data types work and how we can use them in our programs.

Strings

The basic data types also include the type `string` for text variables.

```
# whatever strings you want
(
    'hello world!',
    "hello world!"
)
```

```
('hello world!', 'hello world!')
```

```
# easy way to use single and double quotes
print('she said, "how are you?"')
print("that's right!")
```

```
she said, "how are you?"
that's right!
```

```
(
    'hello' + ' ' + 'world',
    'thinking... ' * 3,
    '*' * 3
)
```

```
('hello world', 'thinking... thinking... thinking... ', '***')
```

Strings belong to the *sequence* type; https://docs.python.org/3.5/library/stdtypes.html#typesseq

This also includes: *ranges*, *tuples* (both immutable), and *lists* (mutable).

All immutable sequences can perform the common sequence operations, and mutable sequences can additionally perform the mutable sequence operations.

```
# indexing
greeting = 'hello there'
(
    greeting[0],
    greeting[6],
    len(greeting),
    greeting[len(greeting)-1]
)
```

```
('h', 't', 11, 'e')
```

```
# negative indexes
(
    greeting[-1],
    greeting[-2],
    greeting[-len(greeting)]
)
```

```
('e', 'r', 'h')
```

```
# "slices"
greeting = 'hello there'
(
    greeting[0:11],
    greeting[0:5],
    greeting[6:11]
)
```

```
('hello there', 'hello', 'there')
```

```
# default slice ranges
(
    greeting[:11],
    greeting[6:],
    greeting[:]
)
```

```
('hello there', 'there', 'hello there')
```

```
# slice "steps"
(
    greeting[0:11:2],
    greeting[::3],
    greeting[6:11:2]
)
```

```
('hlotee', 'hltr', 'tee')
```

```
# negative steps
greeting[::-1]
```

```
'ereht olleh'
```

```
# other sequence ops
greeting = 'hello there!'
(
    greeting.count('e'),
    greeting.index('e'),
    greeting.index('e', 2),
    'e' in greeting,
    'ü' not in greeting,
    min(greeting),
    max(greeting)
)
```

```
(3, 1, 8, True, True, ' ', 't')
```

Strings also have a set of type-specific methods.

Format Strings

It is very easy to replace variables or expressions in strings; this is done with *format strings*.

```
adjective = 'frigid'
adverb = 'hastily'
noun = 'Alfred'
number = 8
verb = 'eat'

sentence = f'It was a {adjective} day when {noun} decided to {adverb} {verb} {number*100}
    lines of code.'

print(sentence)
```

```
It was a frigid day when Alfred decided to hastily eat 800 lines of code.
```

There is also a specific method for it `format`:

```
sentence = 'It was a {} day when {} decided to {} {} {} lines of code.'.format(adjective,
    noun, adverb, verb, number*100)

print(sentence)
```

```
It was a frigid day when Alfred decided to hastily eat 800 lines of code.
```

Lists, Ranges, Dicts, and Sets

In addition to the basic data types, the Python standard library includes several other very powerful data types.

We start with **Ranges**.

Range

A range is a span of numbers between a starting number, an ending number, and optionally a step size.

```
r = range(10)

for i in r:
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

```
r = range(3,15)

for i in r:
    print(i)
```

```
3
4
5
6
7
8
9
10
11
12
13
14
```

```
r = range(1,1000,150)

for i in r:
    print(i)
```

```
1
151
301
451
601
751
901
```

Listen

Lists are a very popular data type in Python because they are very versatile and easy to use. Nevertheless, they come with very powerful functionalities.

```
l = [1,2,3,4,1]
print(l)
```

```
[1, 2, 3, 4, 1]
```

```
# length of a list
len(l)
```

```
5
```

```
# Indexing
l[2]
```

```
3
```

```
# setting a value
l[0] = "Hello"
l
```

```
['Hello', 2, 3, 4, 1]
```

```
l = ["Hello"]
l + ["World"]
```

```
['Hello', 'World']
```

```
l
```

```
['Hello']
```

Lists of other things:

```
l = list(range(6))
l
```

```
[0, 1, 2, 3, 4, 5]
```

```
l = list("European Actuaries")
l
```

```
['E',
 'u',
 'r',
 'o',
 'p',
 'e',
 'a',
 'n',
 ' ',
 'A',
 'c',
 't',
 'u',
 'a',
 'r',
 'i',
 'e',
 's']
```

```
'I love Advanced Programming'.split()
```

```
['I', 'love', 'Advanced', 'Programming']
```

```
'apples, bananas, cats, dogs'.split(',')
```

```
['apples', ' bananas', ' cats', ' dogs']
```

```
# also, strings from lists of strings
'-'.join(['a', 'e', 'i', 'o', 'u'])
```

```
'a-e-i-o-u'
```

```
' 👏 '.join('this is a beautiful day'.split())
```

```
'this 👏 is 👏 a 👏 beautiful 👏 day'
```

Building Lists Simply (List Comprehension)

```
[x for x in range(10)]
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
adjs = ('hot', 'blue', 'quick')
nouns = ('table', 'fox', 'sky')
[adj + ' ' + noun for adj in adjs for noun in nouns]
```

```
['hot table',
 'hot fox',
 'hot sky',
 'blue table',
 'blue fox',
 'blue sky',
 'quick table',
 'quick fox',
 'quick sky']
```

```
# pythagorean triples
n = 50
[(a,b,c) for a in range(1,n)
         for b in range(a,n)
         for c in range(b,n)
         if a**2 + b**2 == c**2]
```

```
[(3, 4, 5),
 (5, 12, 13),
 (6, 8, 10),
 (7, 24, 25),
 (8, 15, 17),
 (9, 12, 15),
 (9, 40, 41),
 (10, 24, 26),
 (12, 16, 20),
 (12, 35, 37),
 (15, 20, 25),
 (15, 36, 39),
 (16, 30, 34),
 (18, 24, 30),
 (20, 21, 29),
 (21, 28, 35),
 (24, 32, 40),
 (27, 36, 45)]
```

Sets

A set is a data structure that represents an *unordered* collection of unique elements. Just like a mathematical set.

```
s = {1, 2, 1, 1, 2, 3, 3, 1}
```

```
s
```

```
{1, 2, 3}
```

```
t = {2, 3, 4, 5}
```

```
s.union(t)
```

```
{1, 2, 3, 4, 5}
```

```
s | t
```

```
{1, 2, 3, 4, 5}
```

```
s.difference(t)
```

```
{1}
```

```
s - t
```

```
{1}
```

```
s.intersection(t)
```

```
{2, 3}
```

```
s & t
```

```
{2, 3}
```

Dicts

A dictionary is a data structure that consists of key-value pairs.

```
d = {
    'Superman':  'Clark Kent',
    'Batman':    'Bruce Wayne',
    'Spiderman': 'Peter Parker',
    'Wonder Woman' : 'Diana Prince',
    'Ironman':   'Tony Stark'
}
```

```
d['Ironman']
```

```
'Tony Stark'
```

```
d['Ironman'] = 'James Rhodes'
```

```
d
```

```
{'Superman': 'Clark Kent',
 'Batman': 'Bruce Wayne',
 'Spiderman': 'Peter Parker',
 'Wonder Woman': 'Diana Prince',
 'Ironman': 'James Rhodes'}
```

```
del d['Ironman']
d
```

```
{'Superman': 'Clark Kent',
 'Batman': 'Bruce Wayne',
 'Spiderman': 'Peter Parker',
 'Wonder Woman': 'Diana Prince'}
```

```
for k in d:
    print(f'{k} => {d[k]}')
```

```
Superman => Clark Kent
Batman => Bruce Wayne
Spiderman => Peter Parker
Wonder Woman => Diana Prince
```

```
for k in d.keys():
    print(f'{k} => {d[k]}')
```

```
Superman => Clark Kent
Batman => Bruce Wayne
Spiderman => Peter Parker
Wonder Woman => Diana Prince
```

```
for v in d.values():
    print(v)
```

```
Clark Kent
Bruce Wayne
Peter Parker
Diana Prince
```

```
for k,v in d.items():
    print(f'{k} => {v}')
```

```
Superman => Clark Kent
Batman => Bruce Wayne
Spiderman => Peter Parker
Wonder Woman => Diana Prince
```

Dictionary comprehensions

```
{e:2**e for e in range(0,100,10)}
```

```
{0: 1,
 10: 1024,
 20: 1048576,
 30: 1073741824,
 40: 1099511627776,
 50: 1125899906842624,
 60: 1152921504606846976,
 70: 1180591620717411303424,
 80: 1208925819614629174706176,
 90: 1237940039285380274899124224}
```

```
{x:y for x in range(3) for y in range(10)}
```

```
{0: 9, 1: 9, 2: 9}
```

Tasks

Please complete the following tasks.

Task 1 (Palindrome Check)

Write a program that checks whether a given string is a palindrome. A palindrome is a word or a sentence that reads the same forwards and backwards (spaces and case are ignored).

Examples of palindromes:

- Madam I m Adam
- A man a plan a canal panama
- Was it a car or a cat I saw

```
# Initialization
text = "Was it a car or a cat I saw"

# Palindrome check
is_palindrome = False
pass

print(f"'{text}' is a palindrome: {is_palindrome}")
```

```
'Was it a car or a cat I saw' is a palindrome: False
```

Task 2 (Square Numbers in a Range)

Write a program that calculates the square numbers in a given range and outputs them as a list. Use list comprehension for this purpose.

```
# Initialization
start = 1
end = 10

# Calculate square numbers
squares = None  # add a suitable list comprehension here

print(squares)
```

```
None
```

Task 3 (Reading Hegel)

In this task, we will read the Phenomenology of Spirit by Georg Wilhelm Friedrich Hegel. Fortunately, "reading" does not always mean "understanding." Your task is to find out:

- How many words did Hegel write?
- How many unique words did Hegel write?
- What is Hegel's most frequently used word?

```
# Run this cell to download the text of Hegel's "The Philosophy of History"
import urllib.request

hegel_url    = 'https://www.gutenberg.org/cache/epub/6698/pg6698.txt'
pdg_text = urllib.request.urlopen(hegel_url).read().decode()
```

Take a look at the text, for example, here is the first description of the Hegelian synthesis:

```
print(pdg_text[7837:8449])
```

How do you get from the individual characters `pdg_text` to the words of the Phenomenology of Spirit? (*Note:* the split function might help.)

```
pdg_words = None # hier geeignet ergänzen
len(pdg_words)
```

And now to the unambiguous words?

```
pdg_words_unique = None
len(pdg_words_unique)
```

And now to the frequency of the individual words. How often was each word written? (*Note:* use a `dict`.)

```
pdg_word_count = {}

for word in pdg_words:
    pass
```

```
print(pdg_word_count)
```

```
{}
```

```
(pdg_word_count["sein"], pdg_word_count["dasein"])
```

Now you just need to sort the dictionary. That's a bit tricky:

```
pdg_word_count_sorted = sorted(pdg_word_count.items(), key=lambda t: t[1], reverse=True)
pdg_word_count_sorted[:10]
```

```
[]
```